

DIPLOMADO EN CIENCIA DE DATOS
Generación 34 · Trabajo final · Módulo V

ChefWise

Un asistente para decidir qué cocinar
y acompañar la preparación

“¿Qué cocino hoy?”

Ciencia de datos aplicada a la cocina de casa

Itzel Guadalupe Camacho Gómez

4 de octubre de 2026

Índice

1	Introducción	3
2	Planteamiento del problema	3
2.1	El problema concreto	3
2.2	Objetivos del proyecto	4
2.3	¿Por qué no basta con un chat de IA general?	4
2.4	Alcance	4
3	Estrategia	4
3.1	Usuarios objetivo y sus necesidades	5
3.2	Accionables	5
3.3	Usabilidad del modelo	6
4	Descripción de los datos	6
4.1	Fuente 1: RecetasDeLaAbuela	7
4.2	Fuente 2: web scraping de Kiwilimón	7
4.3	El catálogo resultante	7
4.4	Calidad de los datos	8
4.5	Relevancia para el problema	8
5	Limpieza de los datos	9
5.1	Unificación de las fuentes	9
5.2	Reparación del texto	9
5.3	Dificultad en tres niveles	10
5.4	Tratamiento de valores extremos en el tiempo	10
5.5	Registros incompletos y duplicados	11
6	Análisis exploratorio de datos	11
6.1	Análisis univariado	11
6.2	Análisis bivariado	12
6.3	Verificación de la etiqueta vegetariana	14
7	Ingeniería de variables	14
7.1	Del ingrediente escrito al ingrediente base	14
7.2	Variables construidas	15
7.3	Dos archivos para dos propósitos	16
7.4	Validaciones automáticas	16
8	Modelación	17
8.1	Recomendador con TF-IDF y similitud coseno	17
8.2	Experimento con embeddings	19
8.3	Evaluación del recomendador	19
8.4	Agrupamiento con K-Means	22
8.5	Asistente generativo con Gemini	23

9	Resolución del problema	23
9.1	Cómo trabajan juntos el recomendador y el asistente	23
9.2	Ejemplo de funcionalidad: Con lo que tengo	24
9.3	Ejemplo de funcionalidad: Sorpréndeme	24
9.4	Recorrido de la interfaz	24
9.5	Cómo responde ChefWise a cada cara del problema	25
10	Conclusiones	25
10.1	Siguientes pasos	26
	Referencias	29

1. Introducción

Resumen

ChefWise es una aplicación web que responde a una pregunta de todos los días: *¿qué cocino hoy?* Recomienda recetas reales de un catálogo de 28,798 platillos en español, a partir de lo que la persona tiene en el refrigerador o de unos cuantos filtros (tiempo, dificultad, país, tipo de comida y alergias). Una vez elegida la receta, un asistente basado en Gemini acompaña la preparación: explica pasos, propone sustituciones, ajusta porciones y recuerda temperaturas seguras. El catálogo se construyó uniendo un conjunto público y recetas obtenidas por web scraping de Kiwilimón; después se limpió, se exploró y se transformó en variables útiles. Se compararon tres enfoques de modelación: TF-IDF con similitud coseno, embeddings y agrupamiento con K-Means. El recomendador publicado, basado en TF-IDF, coloca al menos una receta del mismo tipo de plato entre sus diez sugerencias en el 93 % de las pruebas, contra el 9.5 % de una lista al azar, y responde en unos 3 milisegundos.

Hay un momento, justo antes de cocinar, en el que el problema todavía no es la receta: es *decidir*. El refrigerador tiene cosas, el tiempo es poco y quien está en la cocina no siempre sabe cómo se llama el platillo que podría preparar, ni quiere leer veinte versiones hasta encontrar una. Para quien está empezando a cocinar se suma otra barrera: una instrucción como “sofreír hasta que esté transparente” puede detener toda la preparación.

ChefWise está pensado para ese momento. No inventa platillos: elige uno que ya existe, que alguien ya cocinó y publicó, y lo acompaña con un asistente que conoce la receta que está en pantalla. La meta no es sustituir a quien cocina, sino quitarle la parte más pesada de la decisión y darle confianza mientras prepara la comida.

28,798

recetas listas para
recomendar

2

fuentes de datos: conjunto
público y scraping

93 %

consultas con al menos un
acierto en el top 10

3 ms

para responder una
recomendación

Este documento recorre el proyecto de principio a fin, en el mismo orden en que se construyó: el problema y la estrategia de negocio, la obtención y limpieza de los datos, el análisis exploratorio, la ingeniería de variables, los modelos que se probaron y cómo todo eso se convierte en una aplicación que cualquier persona puede usar. Cada técnica se define antes de aplicarse y cada decisión se acompaña de la evidencia que la sostiene. Los términos técnicos se explican en notas al pie, para que el texto se pueda leer sin conocimientos previos de ciencia de datos.

2. Planteamiento del problema

2.1. El problema concreto

Decidir qué comer parece un problema pequeño, pero se repite todos los días. Tiene tres caras, y las tres pueden aparecer en cualquier momento del día:

1. **El punto de partida.** Hay personas que cocinan poco, o que no tienen ganas, y que aun así tienen que resolver la comida. Conocen pocas recetas, y una receta larga, con un nombre que

no reconocen, es una barrera. Necesitan ingredientes reconocibles y a alguien que traduzca “sofreír hasta que esté transparente” a una instrucción que se pueda seguir.

2. **La despensa.** Lo que está en el refrigerador ya se compró. La pregunta útil es qué platillo real se puede hacer con eso y cuántas cosas extra harían falta. Si la respuesta ignora la despensa, la persona vuelve a la tienda o tira lo que tenía.
3. **La compañía y la decisión.** Cocinar sin nadie al lado deja dudas pequeñas que detienen la preparación: si el fuego está bien, con qué se sustituye lo que no hay, para cuántas personas alcanza, a qué temperatura está seguro el pollo. Además, elegir entre muchas opciones es, para mucha gente, más difícil que seguir una: un buscador con cientos de resultados alarga justo el minuto que se quería acortar.

2.2. Objetivos del proyecto

Para atender esas tres caras se plantearon los siguientes objetivos:

- Construir un **catálogo propio de recetas en español**, limpio y con variables útiles, a partir de fuentes públicas y de web scraping.
- Desarrollar un **sistema de recomendación** que, con los ingredientes disponibles o con filtros sencillos, devuelva recetas reales del catálogo ordenadas por qué tan bien encajan.
- Integrar un **asistente conversacional** que acompañe la receta elegida con explicaciones, sustituciones, ajustes de porciones y datos de seguridad alimentaria.
- Entregar todo en una **aplicación web pública**, intuitiva y sin necesidad de crear una cuenta.

2.3. ¿Por qué no basta con un chat de IA general?

Un modelo de lenguaje¹ de uso general, como ChatGPT, espera una buena pregunta. Formularla ya es parte del problema: quien está indecisa no sabe qué pedir, y el chat no adelanta esa decisión. Además, esos modelos arman la respuesta con material de muchas fuentes y pueden proponer un platillo que suena bien pero que nadie ha cocinado tal cual, con cantidades y tiempos mezclados.

ChefWise funciona al revés. Primero la persona elige con filtros o con ingredientes, sin tener que nombrar el platillo, y el recomendador busca dentro de un catálogo de recetas ya escritas y verificadas. Después, el asistente recibe *esa* receta y la conversación queda acotada a su texto. La ayuda es más concreta porque el universo de respuestas es el de la aplicación.

2.4. Alcance

Para que las expectativas sean claras, conviene decir también lo que la aplicación *no* hace: no inventa recetas nuevas, no calcula información nutricional (las fuentes no la traen) y no garantiza que un platillo esté libre de alérgenos, porque el texto de una receta no describe cómo se fabricó cada producto. Esos límites se explican en la propia interfaz.

3. Estrategia

La estrategia responde tres preguntas: para quién es la aplicación, qué acciones concretas se derivan de los resultados y cómo se logra que el modelo sea fácil de usar. La evidencia de cada

¹Un modelo de lenguaje (LLM, por sus siglas en inglés) es un programa entrenado con enormes cantidades de texto para predecir y generar lenguaje. ChatGPT y Gemini son ejemplos.

decisión está en las secciones de modelación y de resolución; aquí se presenta el plan completo para que el lector sepa desde el principio hacia dónde va el proyecto.

La propuesta en una línea

Elegir una receta que ya existe, con lo que hay en casa o con unos cuantos filtros, y acompañar su preparación con un asistente que conoce esa receta.

3.1. Usuarios objetivo y sus necesidades

La aplicación es para **quien cocina en casa y llega a la comida sin un plan**. Dentro de ese grupo hay necesidades distintas, y el diseño las atiende con pantallas distintas (Cuadro 1).

Cuadro 1: Perfiles de usuario, sus necesidades y la parte de ChefWise que les responde.

Perfil	Qué necesita	Dónde lo encuentra
Principiante o quien cocina de vez en cuando	Pasos cortos y una respuesta directa cuando una técnica no se entiende.	Detalle de la receta y asistente
Quien quiere gastar lo que ya tiene	Escribir sus ingredientes y ver recetas reales que los usen, con una idea de cuántos extras pide cada una.	Con lo que tengo
Quien se queda detenida entre opciones	Pocos controles y una lista breve, con la posibilidad de pedir otras ideas sin rehacer la búsqueda.	Sorpréndeme
Quien cocina con una alergia	Que la aplicación aparte o señale las recetas que contienen el ingrediente marcado.	Mi cocina, filtros y avisos

En el caso de las alergias hay una regla importante: el asistente puede proponer un cambio, pero tiene prohibido decir que una receta es completamente segura. La etiqueta del producto la revisa la persona, porque el texto de una receta no garantiza cómo se fabricó cada ingrediente.

3.2. Accionables

Un accionable es una acción concreta que se toma porque los datos la respaldan. Del análisis y la modelación salen los accionables del Cuadro 2; cada uno indica el hallazgo que lo justifica y la sección donde está la evidencia.

Cuadro 2: Accionables derivados de los resultados del proyecto.

Hallazgo	Acción tomada	Sección
Elegir es la parte difícil, y un chat general no adelanta esa decisión.	Elegir dentro del catálogo, con filtros o con la despena, y enlazar siempre la receta a su página de origen.	9
TF-IDF supera por mucho al azar y, con listas de ingredientes, empata o supera a los embeddings; además cabe en el despliegue.	Publicar TF-IDF como recomendador en todas las pantallas.	8.3
Los embeddings entienden sinónimos (jitomate y tomate), pero pesan más de lo que admite el despliegue.	Dejarlos como experimento documentado y atacar los sinónimos con un diccionario ligero.	8.3
K-Means solo redescubre las etiquetas existentes.	No mostrar grupos; usar las etiquetas del catálogo como fichas de la aplicación.	8.4
Las dudas aparecen cuando la receta ya está elegida.	Activar el asistente sobre la receta abierta, con herramientas de cálculo y una tabla de temperaturas seguras.	8.5
Una de cada cuatro recetas no trae tiempo.	Filtrar por tiempo solo con minutos reales, para no presentar estimaciones como datos.	7
La etiqueta vegetariana pasó una revisión independiente con 0.07 % de alertas.	Usarla como filtro duro y conservar la revisión como alarma automática.	6.3

3.3. Usabilidad del modelo

Un buen modelo que nadie sabe usar no resuelve el problema. Por eso la forma de usarlo se diseñó con cinco principios:

- **Tres caminos, un solo destino.** Desde el inicio la persona puede buscar, pedir que la sorprendan o escribir lo que tiene. Los tres terminan en lo mismo: abrir una receta.
- **No hace falta saber qué pedir.** Quien no sabe por dónde empezar puede elegir “fácil” y un tiempo corto en Sorpréndeme y ya obtiene una lista, sin escribir ninguna pregunta.
- **Sin registro.** Favoritas, historial y alergias se guardan en el navegador, así que la aplicación se puede usar desde el primer segundo.
- **Reglas visibles.** Si un filtro no se pudo cumplir del todo, la aplicación lo dice. Si una receta contiene un alérgeno marcado, el aviso aparece en la misma pantalla.
- **Ayuda en el lugar exacto.** El asistente vive en un botón flotante y responde sobre la receta abierta, en texto claro y breve.

La Sección 9 muestra el recorrido completo de la interfaz con un ejemplo de uso.

4. Descripción de los datos

Para este proyecto, una receta es un texto con nombre, ingredientes y pasos y, cuando la fuente lo trae, dificultad, tiempo, país y valoración. Ese texto tiene que estar en una tabla para poder limpiarlo y modelarlo. Se obtuvo por dos vías.

4.1. Fuente 1: RecetasDeLaAbuela

RecetasDeLaAbuela es un conjunto de datos público publicado en Hugging Face² por la comunidad Somos NLP para trabajar procesamiento de lenguaje en español (Somos NLP, s. f.). El archivo crudo tiene 20,085 filas y 19 columnas, y reúne recetas de varias páginas: RecetasGratis, Yanuq, Cookpad, Directo al Paladar, El Mueble y Vitónica. Sirve como base porque ya trae miles de recetas con ingredientes y preparación, aunque, como se verá, también trae columnas vacías o mal nombradas.

4.2. Fuente 2: web scraping de Kiwilimón

— **Para qué sirve.** El web scraping³ permitió sumar recetas que el conjunto público no traía, sobre todo cocina mexicana de todos los días. Sin este paso el catálogo sería más chico y estaría cargado hacia las páginas españolas.

Kiwilimón (Kiwilimón, s. f.) es un sitio mexicano de recetas de cocina casera. Como el sitio no entrega una tabla lista para descargar, el scraping se hizo con dos programas:

1. El primero lee el índice del sitio, descarga los mapas de direcciones y guarda los enlaces de todas las recetas.
2. El segundo entra a cada enlace con un navegador automatizado, espera a que la página cargue y toma el bloque estructurado de la receta, llamado JSON-LD⁴: título, ingredientes, pasos y valoración.

Leer ese bloque es más estable que adivinar dónde está el texto en el diseño visual de la página, que puede cambiar en cualquier momento. De ahí salieron 9,018 filas útiles.

4.3. El catálogo resultante

Después de la limpieza (Sección 5), el catálogo que muestra la aplicación tiene **28,840 recetas**. El Cuadro 3 y la Figura 1 muestran de dónde viene cada una.

Cuadro 3: Aporte de cada página al catálogo limpio.

Página	Vía de obtención	Recetas	Porcentaje
RecetasGratis	RecetasDeLaAbuela	18,186	63 %
Kiwilimón	Web scraping	9,015	31 %
Otras cinco páginas	RecetasDeLaAbuela	1,639	6 %
Total		28,840	100 %

En cuanto a países, México (10,909) y España (9,381) concentran cerca de siete de cada diez recetas. Les siguen Internacional (4,378) y Perú (1,947), y después una cola de países con muy pocas recetas, como Honduras o Paraguay, que más adelante se agrupan.

²Hugging Face es una plataforma donde la comunidad publica conjuntos de datos y modelos de inteligencia artificial de uso abierto.

³El web scraping es la técnica de leer páginas web de forma automática con un programa y guardar la información que interesa en una tabla.

⁴JSON-LD es un recuadro de datos que las propias páginas publican para que otras máquinas, como los buscadores, entiendan su contenido; por ejemplo, “esto es una receta, este es su título y estos sus ingredientes”.

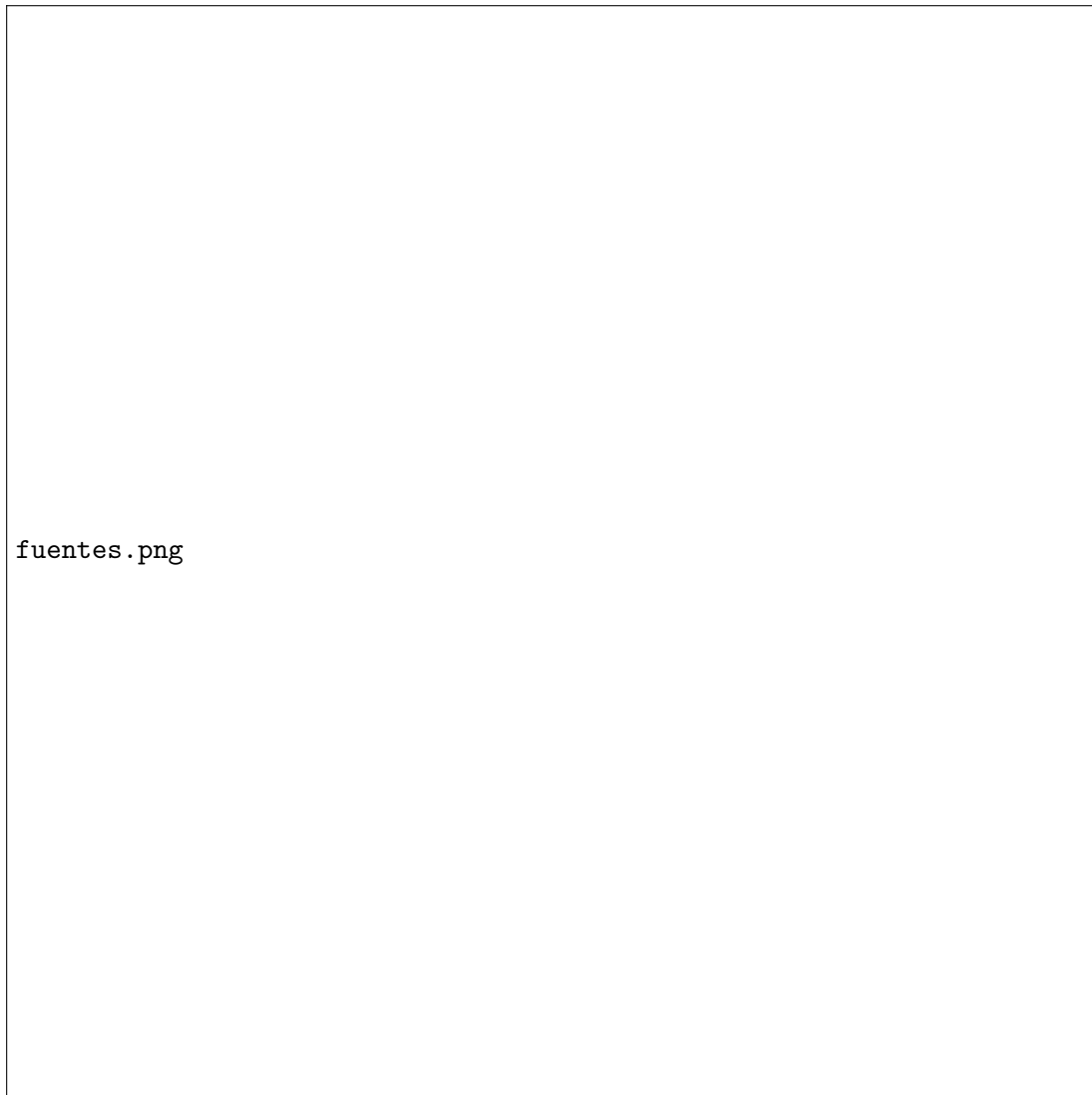


Figura 1: Recetas del catálogo limpio según la página de origen. Kiwilimón entra por scraping; el resto llega dentro de RecetasDeLaAbuela.

4.4. Calidad de los datos

— **Para qué sirve.** Revisar la calidad antes de modelar evita enseñarle al modelo un error de la página como si fuera una característica del platillo. Si una marca, un anuncio o un tiempo imposible se quedan en el texto, el recomendador los usa para decidir que dos recetas se parecen.

Los datos crudos presentaron varios problemas, resumidos en el Cuadro 4.

Un hallazgo cambió la forma de limpiar: los valores faltantes dependen de la *página*, no de la receta (Figura 2). Kiwilimón casi siempre trae dificultad, tiempo y valoración; Yanuq no trae ninguno; Cookpad, Directo al Paladar, El Mueble y Vitónica traen tiempo pero omiten los otros dos. Si se borrara toda receta con un hueco, desaparecerían fuentes enteras. Por eso el hueco se conserva y se marca.

4.5. Relevancia para el problema

Los datos son adecuados para el problema por tres razones: están en español y con fuerte presencia de cocina mexicana, que es la cocina de casa que se quería cubrir; traen la lista de ingredientes, que es justo lo que se compara con la despensa; y conservan el enlace a la página

Cuadro 4: Principales problemas de calidad encontrados en los datos crudos.

Problema	Ejemplo
Columnas vacías	La columna de calorías de RecetasDeLaAbuela no tiene ningún valor.
Columnas mal nombradas	“Porciones” trae una lista de ingredientes en 18,187 filas y un número de comensales solo en 632. “Tiempo” guarda el tipo de plato (postre, desayuno).
Letras rotas	limón guardado como limÃ³n.
Texto que no es receta	Párrafos de presentación del blog, “sígueme en Instagram”, marcas comerciales.
Registros incompletos	210 recetas sin ingredientes y 211 sin pasos.
Filas mal formadas	En Kiwilimón, una línea pega dos recetas y trae 17 campos en lugar de 9.
Tiempos imposibles	4,652 cocciones en cero minutos, 244 tiempos totales de más de 12 horas, y pays con 30 o 60 “horas” de preparación.

original, de modo que cada recomendación es una receta real que la persona puede verificar. Cumplen además la condición del diplomado de no usar Kaggle como fuente.

5. Limpieza de los datos

La limpieza deja una receta que se puede leer. Ninguna de sus reglas reescribe el platillo: se corrige el formato y se descarta lo que no es un dato.

Antes y después

Antes: 2 tazas de azÃ³car | Coca Cola | Para 4 personas | Sígueme en Instagram

Después: 2 tazas de azúcar | refresco de cola

El azúcar se puede leer otra vez. La marca se vuelve el producto genérico, para que la receta siga siendo cocinable. La nota de porciones y la autopromoción se eliminan, porque no se compran ni se cocinan.

5.1. Unificación de las fuentes

Las dos fuentes se llevaron al mismo esquema de columnas y se apilaron en una sola tabla. En Kiwilimón, en lugar de escribir una regla especial para la línea que pegaba dos recetas, se aplicó una regla general: se descarta cualquier fila que no tenga exactamente nueve campos. Así el proceso no se rompe si una extracción futura falla de otra forma.

5.2. Reparación del texto

Se repararon las letras rotas, que aparecen cuando un texto codificado en UTF-8⁵ se lee con otra codificación. También se quitaron emojis, restos de HTML, marcas comerciales y frases de autopromoción. Las listas de ingredientes y pasos pasaron a un texto separado por barras verticales, y a los nombres se les quitó el “Cómo hacer” del inicio, que es una fórmula del título y no parte del platillo.

⁵UTF-8 es la forma estándar de guardar letras en una computadora. Cuando un archivo UTF-8 se lee con otra codificación, las letras con acento se convierten en símbolos extraños, como Ã³ en lugar de ó.



Figura 2: Porcentaje de valores faltantes por página de origen. El hueco sigue a la fuente, no a la receta.

5.3. Dificultad en tres niveles

Cada página describe la dificultad a su manera. Se unificó en tres niveles: **fácil**, **intermedio** y **reto**. Tres niveles bastan para el filtro de la aplicación, y conservar las diez formas en que cada sitio dice “fácil” obligaría a la persona a conocer el vocabulario de cada página.

5.4. Tratamiento de valores extremos en el tiempo

Los tiempos se convirtieron a minutos, que es la unidad del deslizador de la aplicación, y se aplicaron tres reglas:

- Un **cero** se trata como dato ausente: una receta de cero minutos no existe, es un campo vacío mal capturado.
- En Kiwilimón, una preparación de entre 25 y 96 “horas” en un platillo que no fermenta se interpreta como **minutos escritos en la casilla de horas**.
- Por encima de eso, o con más de 12 horas de tiempo total, el campo se deja vacío, porque no describe una comida del mismo día.

Con estas reglas, el 76 % del catálogo conserva un tiempo confiable.

5.5. Registros incompletos y duplicados

Se eliminaron las recetas sin nombre, sin ingredientes o sin pasos, porque no se pueden recomendar ni cocinar. Al final se quitaron los duplicados. El archivo que muestra la aplicación cierra con **28,840 recetas**.

6. Análisis exploratorio de datos

El análisis exploratorio de datos (EDA, por sus siglas en inglés) tiene como objetivo conocer los datos antes de modelarlos: cómo se distribuyen, qué falta y qué se relaciona con qué. Se usan dos enfoques: **univariado**, que mira una variable a la vez, y **bivariado**, que mira pares de variables. En este proyecto cada gráfica responde una pregunta práctica: *¿este campo sirve para recomendar, o es un hueco, un error o una repetición de otro?*

6.1. Análisis univariado

6.1.1. Tiempo de preparación

El Cuadro 5 resume las estadísticas descriptivas del tiempo total, calculadas sobre las 21,971 recetas que lo reportan.

Cuadro 5: Estadísticas descriptivas del tiempo total de preparación.

Medida	Valor
Recetas con tiempo conocido	21,971
Mediana	40 min
Promedio	52 min
Percentil 90	90 min
Máximo	720 min
Recetas de más de 120 minutos	6.5 %

La mediana⁶ es de 40 minutos y el percentil 90⁷ de 90 minutos. El promedio (52 minutos) es mayor que la mediana porque unas pocas recetas muy largas lo jalen hacia arriba. Esa cola es real: panes, marinados y preparaciones que se dejan de un día para otro. La Figura 3 muestra la masa principal; las recetas de más de 120 minutos quedan fuera para que la escala se pueda leer.

Qué significa para la aplicación: “algo rápido” es un filtro con sentido, porque hay suficientes recetas cortas para devolver una lista. Las recetas largas no se recortan, porque a veces el tiempo largo es parte del platillo.

6.1.2. Dificultad

De las 22,797 recetas con dificultad conocida, 18,088 son fáciles (79 %), 4,280 intermedias (19 %) y 429 de reto (2 %). Se intentó estimar la dificultad faltante a partir del tiempo y del número de pasos, pero como la clase “fácil” es tan grande, la estimación respondía “fácil” casi siempre. Un dato inventado que siempre dice lo mismo no ayuda a filtrar, así que la dificultad ausente se deja vacía. La buena noticia para el usuario principiante es que tiene miles de recetas fáciles disponibles.

⁶La mediana es el valor que queda justo a la mitad cuando los datos se ordenan: la mitad de las recetas tarda menos de 40 minutos y la otra mitad, más.

⁷El percentil 90 es el valor por debajo del cual queda el 90 % de los datos: nueve de cada diez recetas tardan 90 minutos o menos.

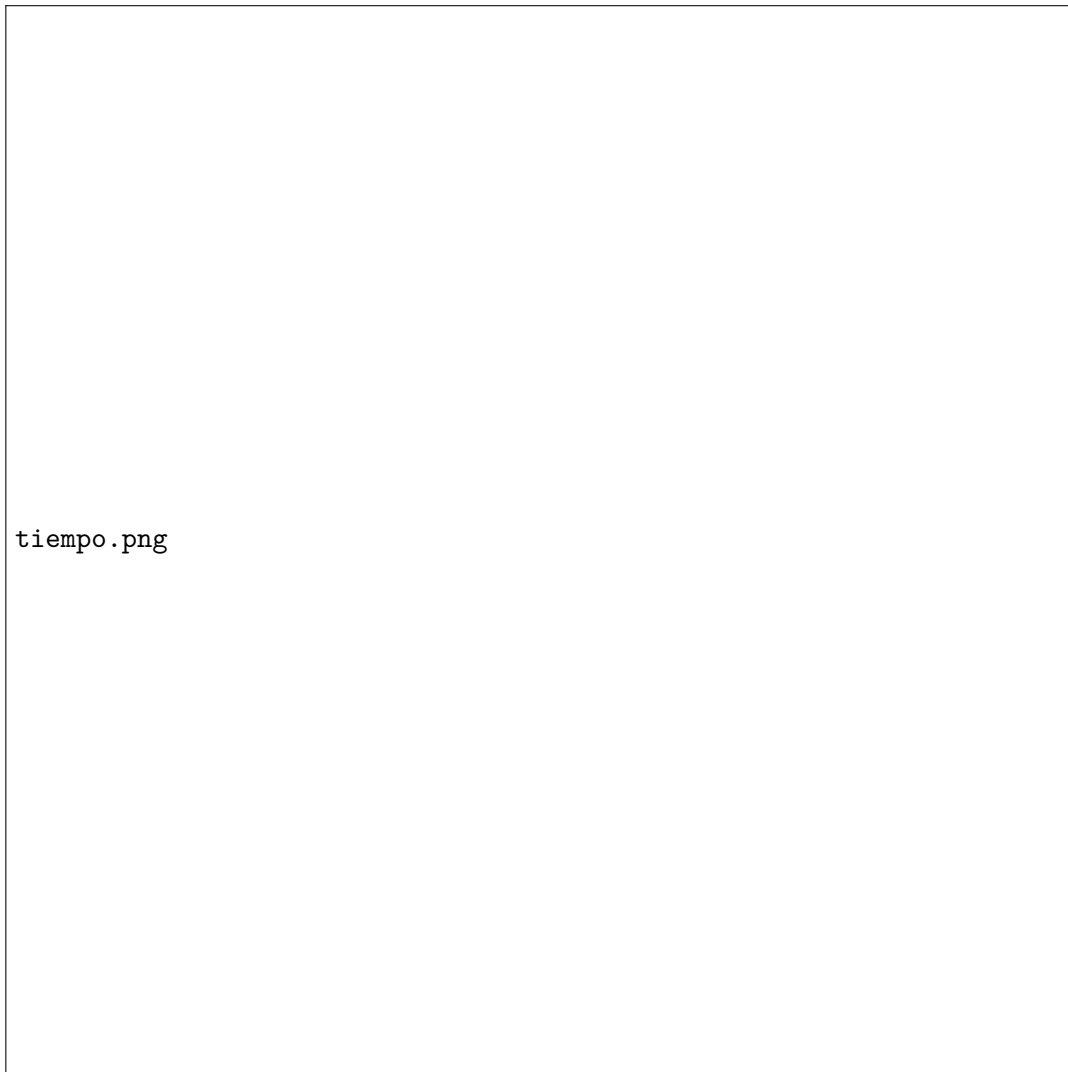


Figura 3: Distribución del tiempo de las recetas que tardan hasta 120 minutos. La línea marca la mediana de 40 minutos.

6.1.3. Etiquetas

Hay 16 etiquetas y una receta puede tener varias. La Figura 4 muestra su frecuencia: “comida” y “vegetariana” dominan, mientras que pavo y cordero son grupos pequeños pero reconocibles. Casi todas las recetas tienen al menos una etiqueta y la mayoría tiene un momento del día (desayuno, comida, cena, postre o bebida). Estas etiquetas son las fichas que la persona toca en Sorpréndeme, por eso era importante verificar que estuvieran bien puestas.

6.1.4. Valoración

La valoración en estrellas solo existe en RecetasGratis y en Kiwilimón, y falta en el 23 % del catálogo. En Kiwilimón el promedio ronda 4.7 con mediana de 5; en RecetasGratis el promedio ronda 4.

6.2. Análisis bivariado

6.2.1. Correlación de Spearman

Para saber si las variables numéricas se mueven juntas se usó la correlación de Spearman (Spearman, 1904). Esta medida ordena los valores de cada variable de menor a mayor y compara



Figura 4: Frecuencia de las 16 etiquetas del catálogo. Una receta puede sumar en más de una.

esos lugares, en lugar de los valores originales. Se calcula como:

$$\rho = 1 - \frac{6 \sum_{i=1}^n d_i^2}{n(n^2 - 1)}$$

donde:

- d_i es la diferencia entre el lugar que ocupa la receta i en una variable y el que ocupa en la otra;
- n es el número de recetas.

El resultado va de -1 a 1 . Un valor cercano a 1 indica que cuando una variable sube, la otra también tiende a subir; cercano a 0 , que no hay relación. Spearman tiene la ventaja de no exigir que la relación sea una línea recta y de no dejarse llevar por valores extremos, como las recetas de 12 horas.

Entre tiempo, número de ingredientes, número de pasos y dificultad, la correlación es positiva y queda casi siempre entre **0.2 y 0.4**: más ingredientes suelen ir con más tiempo, pero la relación es moderada. Las pocas recetas de reto se distinguen porque tardan más, no porque usen muchos más ingredientes.

Qué significa para la aplicación: cada variable aporta algo propio. Fundirlas en un solo

índice y descartar las originales haría perder filtros como “30 minutos” o “sin horno”. Por eso el índice de esfuerzo de la siguiente sección se construye como un resumen adicional y las variables originales se conservan.

6.2.2. Valoración contra fuente

Al cruzar la valoración con la página de origen aparece un patrón claro: la diferencia de estrellas es de la página, no del platillo. Una receta de 4.2 en un sitio puede estar mejor considerada que una de 4.8 en el otro. Además, la valoración casi no se relaciona con el tiempo, los ingredientes ni la dificultad. Por eso se muestra como dato de la fuente, pero *no* se usa para ordenar recomendaciones: hacerlo mezclaría dos escalas distintas.

6.3. Verificación de la etiqueta vegetariana

La etiqueta vegetariana es la más delicada: si falla, la aplicación le entrega carne a quien pidió lo contrario. Se revisó con una lista de palabras de origen animal *distinta* de la que asigna la etiqueta, para no preguntarle al mismo código si se equivocó. De 13,584 recetas revisadas, solo 9 (0.07 %) activaron una palabra de la lista, y al leer el título se confirmó que la etiqueta era correcta. Esa revisión se quedó como alarma automática: si una versión futura del catálogo la rompe, el proceso se detiene.

Lo que el EDA decidió

- El tiempo sirve para filtrar, pero solo cuando viene de la fuente.
- La dificultad faltante no se imputa: la estimación siempre diría “fácil”.
- La valoración no ordena recomendaciones, porque cada página usa su propia escala.
- Las variables numéricas se conservan por separado, porque se relacionan solo de forma moderada.
- Las etiquetas son confiables y se pueden usar como filtros.

7. Ingeniería de variables

La ingeniería de variables construye columnas nuevas que el dato crudo no trae, pero que el modelo o los filtros sí necesitan. Cada columna de esta sección existe porque una pregunta de la aplicación no se podía responder con el texto original.

7.1. Del ingrediente escrito al ingrediente base

Los ingredientes llegan como un solo texto, por ejemplo **2 tazas de harina | 1 cucharadita de sal | 3 huevos**. Para compararlos con la despensa hay que separarlos, quitar los títulos de sección (“para la salsa”) y las notas de porciones, y quedarse solo con el nombre: **harina, sal, huevo**.

Después se llevó cada ingrediente a su forma singular, un proceso parecido a la lematización⁸. En lugar de usar una regla ciega como “quítale la s”, el singular se decidió con el propio catálogo: **nueces** pasa a **nuez** porque **nuez** aparece en otras recetas, y **anís** se queda igual porque **aní** no

⁸La lematización consiste en llevar cada palabra a su forma base o de diccionario; por ejemplo, “cocinábamos” a “cocinar” o “nueces” a “nuez”.

existe. Así se evita que el modelo vea palabras como **nuece** o **frijole**, que ninguna persona va a escribir en su despena.

7.2. Variables construidas

7.2.1. Índice de esfuerzo

El índice de esfuerzo (`effort_score`) mide qué tan trabajosa es una receta, incluso cuando la página no puso dificultad. Se calcula como el promedio de dos posiciones relativas:

$$\text{esfuerzo}_i = \frac{\text{rango}(\text{ingredientes}_i) + \text{rango}(\text{pasos}_i)}{2}$$

donde $\text{rango}(\cdot)$ es la posición de la receta i entre todas las del catálogo, expresada entre 0 y 1. Un valor de 0.9 significa “esta receta tiene más ingredientes y pasos que el 90 % del catálogo”.

7.2.2. Tiempo conocido y tiempo estimado

El tiempo faltante se estima solo en una columna de uso numérico, y una columna aparte, `time_known`, dice si el minuto viene de la fuente. El filtro “máximo 30 minutos” usa únicamente el tiempo real. Así, una receta sin tiempo nunca aparece como si durara media hora.

7.2.3. Variables indicadoras

Cada etiqueta se convirtió en una columna de ceros y unos⁹. Del texto de los pasos se leyeron además cuatro banderas: si la receta es picante, si usa horno, si lleva fritura y si necesita licuadora. Sirven para filtros como “sin horno” sin pedirle al modelo que lea todo el procedimiento.

7.2.4. País agrupado

Los países con muy pocas recetas se agruparon en la categoría **Otro**, porque con unas cuantas recetas un modelo no alcanza a “aprender” esa cocina. La columna original se conserva para el filtro de la página.

7.2.5. El texto que lee el recomendador

La columna `recipe_text` junta en un solo texto el nombre, los ingredientes base, las etiquetas y el país. Lo que se dejó fuera es tan importante como lo que entró:

- **Las cantidades**, porque “2 tazas de harina” y “3 tazas de harina” serían palabras distintas para el mismo platillo.
- **Los pasos**, porque palabras como “mezcla” u “hornea” aparecen en casi todo el catálogo y harían parecerse recetas que no se parecen.
- **La palabra “internacional”**, porque la fuente la usa cuando no sabe el origen.
- **Adjetivos de anuncio** como “fácil”, “casero” o “delicioso”: un tiramisú y una torta de chocolate no deberían parecerse solo porque los dos títulos dicen “paso a paso”.

7.2.6. Clave estable

`recipe_key` es un código calculado a partir de la fuente, el enlace y el nombre de la receta¹⁰. A diferencia de un número consecutivo, no cambia si el catálogo se reordena, así que es la clave

⁹A esto se le llama codificación *one-hot*: una columna por categoría, con 1 si la receta pertenece a ella y 0 si no. Los modelos leen números, no frases como “Comida | Pollo”.

¹⁰Técnicamente es un *hash*: una función que convierte un texto en un código corto y siempre da el mismo código para el mismo texto.

que puede guardar un favorito.

Con estas variables se puede describir la receta típica del catálogo: **8 ingredientes y 9 pasos** (medianas). El 36 % de las recetas está marcada como rápida, es decir, tarda 30 minutos o menos con un tiempo que sí venía en la fuente.

7.3. Dos archivos para dos propósitos

El resultado son dos tablas unidas por `recipe_id`. `recetas_limpias.csv` (28,840 filas) es la que ve la persona: necesita leer “2 tazas de harina” y el paso completo. `recetas_modelo.csv` (28,798 filas y 43 columnas) es la que leen los modelos. Las 42 recetas que faltan en la segunda tienen menos de dos ingredientes limpios: se pueden consultar, pero no entran al recomendador, porque con un solo ingrediente no hay un platillo que recomendar.

Cuadro 6: Columnas del archivo que ve la persona.

Columna	Para qué está
<code>recipe_id</code>	Número de la receta en esta versión del catálogo.
<code>name, ingredients, instructions</code>	Lo que se lee en pantalla: nombre, ingredientes con cantidad y pasos.
<code>tags</code>	Fichas de la aplicación (desayuno, pollo, vegetariana). Puede haber varias.
<code>difficulty</code>	Fácil, intermedio o reto. Puede faltar.
<code>total_time_min</code>	Minutos totales, para el filtro de tiempo. Puede faltar.
<code>country</code>	País, para filtrar. Puede faltar.
<code>rating</code>	Estrellas de la página, de 0 a 5. Se muestra, pero no ordena.
<code>source, url</code>	Página de origen y enlace, para volver a la fuente.

Cuadro 7: Principales variables del archivo que leen los modelos (43 columnas en total).

Columna	Qué significa
<code>recipe_key</code>	Clave estable calculada con el contenido.
<code>ingredients_core</code>	Ingredientes sin cantidad ni unidad; es lo que se compara con la despensa.
<code>n_ingredients, n_steps</code>	Número de ingredientes y de pasos.
<code>effort_score</code>	Índice de esfuerzo entre 0 y 1.
<code>difficulty_known, time_known</code>	Indican si el dato vino de la fuente o es un hueco.
<code>is_quick</code>	30 minutos o menos, con tiempo real.
<code>country_group</code>	País, con los de pocas recetas agrupados en Otro.
<code>tag_*</code>	Una columna de 0 y 1 por etiqueta.
<code>is_spicy, needs_oven, needs_frying, needs_blender</code>	Banderas leídas del texto de la receta.
<code>recipe_text</code>	Texto único por receta que lee el recomendador.

7.4. Validaciones automáticas

Antes de guardar, el notebook comprueba que los identificadores sean únicos, que no haya textos vacíos, que las columnas de 0 y 1 coincidan con las etiquetas, que no haya plurales mal cortados y que toda receta del modelo exista en el archivo que ve la persona. Si algo falla, el proceso se detiene. Esta comprobación garantiza que la tabla que se muestra y la tabla que se modela hablan de las mismas recetas.

8. Modelación

En ciencia de datos se distingue entre aprendizaje **supervisado**, en el que el modelo aprende de ejemplos con la respuesta correcta ya marcada, y **no supervisado**, en el que solo recibe los datos y debe encontrar parecidos o grupos por sí mismo. Para entrenar un recomendador supervisado harían falta datos de comportamiento, como clics o recetas cocinadas por personas reales, y este proyecto no los tiene. Por eso la modelación es **no supervisada**, complementada con un modelo **generativo** para el asistente.

Se probaron cuatro modelos, y cada uno responde una pregunta distinta (Cuadro 8).

Cuadro 8: Modelos probados, la pregunta que responde cada uno y su estado final.

Modelo	Pregunta que responde	Estado
TF-IDF + coseno	¿Qué receta del catálogo se parece a lo que escribió la persona?	En la aplicación
Embeddings	La misma pregunta, con una noción de parecido basada en significado.	Experimento
K-Means	¿El catálogo se divide en grupos que las etiquetas no muestran?	Descartado
Gemini	¿Cómo ayudo a cocinar la receta que ya se eligió?	En la aplicación

8.1. Recomendador con TF-IDF y similitud coseno

— **Para qué sirve.** TF-IDF convierte el texto de cada receta en números para poder compararlas (Salton y Buckley, 1988). Sirve para ordenar el catálogo cuando la persona escribe ingredientes o marca un antojo, sin inventar una receta nueva.

8.1.1. ¿Qué es TF-IDF?

Una computadora no entiende que “chipotle” es un chile; solo ve una palabra. TF-IDF (*Term Frequency – Inverse Document Frequency*) le asigna a cada palabra un peso que **sube** si aparece en la receta y **baja** si aparece en muchísimas recetas. La idea es que “chipotle” dice más de un platillo que “sal”, porque la sal está en todas partes. El peso se calcula así:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \text{idf}(t), \quad \text{idf}(t) = \ln\left(\frac{1 + N}{1 + \text{df}(t)}\right) + 1$$

donde:

- $\text{tf}(t, d)$ es cuántas veces aparece la palabra t en la receta d ;
- N es el número total de recetas;
- $\text{df}(t)$ es el número de recetas que contienen la palabra t .

Ejemplo: un catálogo de juguete con tres recetas

Suponga un catálogo con tres recetas: **A** tinga (sal, pollo, chipotle), **B** pollo asado (sal, pollo) y **C** pastel (sal). Con $N = 3$, el peso idf de cada palabra es:

Palabra	Recetas que la contienen	idf	Lectura
sal	3	1.00	no distingue a nadie
pollo	2	1.29	acerca la tinga al pollo asado
chipotle	1	1.69	lo que hace única a la tinga

La palabra rara pesa más que la palabra que está en todas partes. Es exactamente lo que se busca.

El modelo real hace esta cuenta sobre **5,436 palabras y 28,798 recetas**, con cinco ajustes que salieron de observar el comportamiento del texto:

1. **Solo palabras sueltas.** Una primera versión usaba también pares de palabras (n-gramas¹¹), pero como los ingredientes van seguidos en el texto, se formaban pares sin sentido como **amarillo lechuga**, que recibían un peso alto por ser raros.
2. **Se ignoran palabras que aparecen en una sola receta**, que suelen ser errores de escritura.
3. **Se ignoran palabras que aparecen en más de la mitad de las recetas**, como sal.
4. **Frecuencia amortiguada:** si una palabra se repite mucho en la misma receta, su peso crece menos que el conteo, para que una receta no gane solo por decir “pollo” cinco veces.
5. **Minúsculas y sin acentos**, para que “plátano” y “platano” coincidan con lo que escribe la persona.

8.1.2. Similitud coseno

Una vez que cada receta es una lista de pesos, es decir, un vector¹², se necesita una forma de medir qué tan parecidas son dos recetas. Para eso se usa la similitud coseno (Manning et al., 2008):

$$\text{similitud}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

donde $A \cdot B$ es el producto punto de los dos vectores (multiplicar peso por peso y sumar) y $\|A\|$ es el largo del vector A . El resultado va de 0 a 1: si dos recetas apuntan en la misma dirección, se parecen y el valor se acerca a 1; si no comparten ninguna palabra con peso, el valor es 0. El coseno mira la dirección y no el largo, así que una receta de treinta ingredientes no le gana a una de seis solo por ser más larga.

Continuación del ejemplo

Con los pesos anteriores, las similitudes entre las tres recetas quedan así:

Par de recetas	Similitud coseno
Tinga – Pollo asado	0.69
Pollo asado – Pastel	0.61
Tinga – Pastel	0.43

La tinga se parece más al pollo asado que al pastel, como esperaría cualquier persona. En el modelo real la sal ni siquiera contaría, por el ajuste 3, y la diferencia sería todavía mayor.

¹¹Un n-grama es una secuencia de n palabras seguidas. “Chile chipotle” es un bigrama, es decir, un 2-grama.

¹²Un vector es una lista ordenada de números. Se puede imaginar como una flecha que apunta en cierta dirección dentro de un espacio con tantas dimensiones como palabras tiene el vocabulario.

8.1.3. Cómo se usa en la aplicación

Primero, los **filtros duros** (dificultad, tiempo, país, etiqueta y alergia) acotan el conjunto de recetas: si la persona pide “fácil y 30 minutos”, una receta intermedia no entra aunque su texto se parezca. Después, TF-IDF **ordena** a las recetas que sí cumplen. Una receta sin ninguna palabra en común con la consulta no entra a la lista, porque no usa lo que la persona dijo tener.

En cuanto a costo, la matriz de 28,798 recetas por 5,436 palabras es dispersa¹³: ocupa cerca de 6.1 MB en disco, se construye en una fracción de segundo y responde un top 10 en unos 3 milisegundos.

8.2. Experimento con embeddings

— **Para qué sirve.** Un embedding¹⁴ coloca cada texto en un “mapa” de 384 números en el que los textos con significado parecido quedan cerca, aunque no compartan palabras (Wang et al., 2024). Sirve para reconocer que jitomate y tomate, o dos ceviches con distinto pescado, son el mismo tipo de cosa.

TF-IDF solo junta recetas que comparten letras: para él, “jitomate” y “tomate” tienen similitud 0, igual que “papa” y “patata”. Un embedding ya vio esas palabras en contextos parecidos y las pone cerca. Se eligió el modelo `multilingual-e5-small` porque entiende español, está diseñado para buscar textos parecidos y lee hasta 512 tokens¹⁵, suficiente para cada receta. Calcular los vectores de las 28,798 recetas tomó 26 segundos en una GPU¹⁶. El archivo de vectores ocupa 44 MB y el modelo que los produce, 488 MB.

8.3. Evaluación del recomendador

8.3.1. Diseño de la prueba

Sin clics de usuarios, hace falta una regla de acierto escrita antes de ver los resultados: **una recomendación es correcta si es del mismo tipo de plato que la receta de partida** (tacos con tacos, sopa con sopa). El tipo se lee del nombre, entre 32 tipos con al menos 20 recetas cada uno. De cada tipo se sortearon 10 recetas con una semilla fija¹⁷, para un total de 320 consultas, las mismas para todos los modelos. Se hicieron dos pruebas:

- **Prueba A.** La consulta es la receta completa. Imita “me gustó esta, ¿qué otra se parece?”.
- **Prueba B.** La consulta es solo la lista de ingredientes. Imita “tengo esto, ¿qué cocino?”, es decir, la pantalla Con lo que tengo.

8.3.2. Métricas

Se usaron cuatro métricas de recuperación de información (Manning et al., 2008; Järvelin y Kekäläinen, 2002), todas calculadas sobre las $K = 10$ primeras recomendaciones:

¹³Una matriz dispersa es una tabla en la que casi todos los valores son cero. Se guarda solo lo que no es cero, lo que ahorra mucho espacio.

¹⁴Un embedding es una representación de un texto como una lista de números, calculada por un modelo de lenguaje que ya fue entrenado con millones de textos. Textos con significado parecido quedan cerca entre sí.

¹⁵Un token es un pedazo de texto (una palabra o parte de ella) con el que trabajan los modelos de lenguaje.

¹⁶Una GPU, o tarjeta gráfica, es un procesador especializado en hacer muchas operaciones a la vez, lo que acelera los modelos de IA.

¹⁷Una semilla fija hace que el sorteo aleatorio sea siempre el mismo, para que cualquier persona pueda repetir el experimento y obtener los mismos resultados.

- **Precision@K**: fracción de aciertos dentro de la lista. Si 5 de 10 son del mismo tipo, vale 0.5.

$$\text{Precision@K} = \frac{\text{aciertos en las primeras } K}{K}$$

- **HitRate@K**: vale 1 si hubo al menos un acierto en la lista y 0 si no. Responde “¿la lista le sirvió de algo?”.
- **NDCG@K** (*Normalized Discounted Cumulative Gain*): premia que los aciertos aparezcan arriba. Acertar en el lugar 1 vale más que acertar en el 10.

$$\text{DCG@K} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{DCG ideal@K}}$$

donde rel_i vale 1 si la receta en el lugar i es un acierto y 0 si no, y el DCG ideal es el que se obtendría con todos los aciertos al principio.

- **Recall@K**: fracción de todos los aciertos posibles que entraron en la lista. Sale bajo por diseño: puede haber 865 sopas en el catálogo y solo se muestran 10. Sirve para comparar modelos entre sí.

La referencia mínima es **recomendar al azar**. Para saber si una diferencia entre modelos es estable se usó un *bootstrap*¹⁸ de 10,000 repeticiones (Efron y Tibshirani, 1993).

8.3.3. Resultados

Cuadro 9: Métricas con $K = 10$, promedio de las 320 consultas. Más alto es mejor.

Prueba	Modelo	Precision@10	HitRate@10	NDCG@10
A: receta completa	TF-IDF	0.475	0.931	0.492
	Embeddings	0.552	0.963	0.576
	Azar	0.011	0.095	0.011
B: solo ingredientes	TF-IDF	0.231	0.691	0.241
	Embeddings	0.221	0.647	0.237
	Azar	0.011	0.095	0.011

Los resultados (Cuadro 9 y Figura 5) se leen así:

- **Los dos modelos aprenden algo real.** En la prueba A, la precisión de TF-IDF es unas cuarenta veces la del azar, y en el 93 % de las consultas hay al menos un acierto en el top 10.
- **Con la receta completa, ganan los embeddings.** Suben la Precision@10 de 0.475 a 0.552, con un intervalo de confianza del 95 % por encima de cero, y ganan en 25 de los 32 tipos de plato. Un ejemplo lo explica bien: ante un ceviche de merluza, TF-IDF se queda en la palabra “merluza” y propone otros platos de merluza, mientras que el embedding reconoce que es un ceviche y propone ceviches.
- **Con solo ingredientes, empatan, con ligera ventaja de TF-IDF.** Los intervalos cruzan el cero, así que la diferencia no es concluyente. Al quitar 44 consultas cuya lista de ingredientes todavía nombraba el tipo de plato, TF-IDF queda por delante. Con una lista de ingredientes, la palabra exacta es una buena señal.

¹⁸El bootstrap consiste en volver a sortear muchas veces las consultas, con repetición, y recalcular la métrica cada vez. Si en el 95 % de los sorteos la diferencia sigue a favor de un modelo, la ventaja no es casualidad.



Figura 5: TF-IDF y embeddings frente al azar en las dos pruebas.

8.3.4. Costo y decisión

Los dos modelos responden rápido, así que lo que decide es el espacio (Cuadro 10). Vercel, la plataforma donde está publicada la aplicación, limita el tamaño de la función que atiende las peticiones, y el modelo de embeddings no cabe. Por eso **la aplicación publicada usa TF-IDF**. La prueba B respalda la decisión: la pantalla que parte de la despensa, que es el corazón del problema, es justamente donde TF-IDF se defiende mejor.

La regla de acierto tiene un límite que conviene decir: unas flautas, cuando la consulta era de tacos, no cuentan como acierto aunque en la cocina se parezcan. “Mismo tipo de plato” tampoco equivale a “a esta persona le sirvió”. La regla sirvió para comparar modelos con la misma vara, pero no sustituye una prueba con personas cocinando.

Cuadro 10: Costo de tener cada modelo listo para responder.

	TF-IDF	Embeddings
Números por receta	5,436, casi todos en cero	384, todos con valor
Tiempo de construcción	0.2 s	26 s en GPU
Tamaño en disco	6.2 MB	44 MB + modelo de 488 MB
De un texto al top 10	2.9 ms	9.7 ms

8.4. Agrupamiento con K-Means

— **Para qué sirve.** K-Means divide las recetas en grupos de recetas parecidas sin que nadie le diga cuáles son (MacQueen, 1967; Pedregosa et al., 2011). Sirve para preguntar si el catálogo tiene una estructura escondida que valga la pena mostrar en la aplicación.

8.4.1. ¿Cómo funciona?

El algoritmo sigue cuatro pasos:

1. Se elige un número de grupos k y se colocan k centros al azar.
2. Cada receta se asigna al centro más cercano.
3. Cada centro se mueve al promedio de las recetas de su grupo.
4. Se repiten los pasos 2 y 3 hasta que los grupos ya no cambian.

Para el agrupamiento entraron 27 variables numéricas: ingredientes, pasos, minutos, etiquetas, banderas de técnica y dificultad. No entró el texto, porque el parecido por palabras ya lo cubren los otros modelos, ni el país, porque México y España dominarían los grupos. Los conteos y los minutos pasaron por un logaritmo y después todas las variables se llevaron a la misma escala; sin ese paso, 400 minutos aplastarían al 0 o 1 de “lleva horno”.

8.4.2. ¿Cuántos grupos?

Se probaron valores de k de 2 a 15 con tres métricas. La principal es la **silueta** (Rousseeuw, 1987), que para cada receta i compara qué tan cerca está de su grupo contra el grupo vecino:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

donde $a(i)$ es la distancia promedio de la receta a las de su propio grupo y $b(i)$ la distancia promedio al grupo vecino más cercano. Un valor cercano a 1 indica un corte limpio; cercano a 0, que la receta está en la frontera; negativo, que encajaría mejor en otro grupo. Se revisaron también los índices de Calinski-Harabasz (Caliński y Harabasz, 1974), que premia grupos compactos y separados, y de Davies-Bouldin (Davies y Bouldin, 1979), que debe ser bajo.

Las tres métricas no coincidieron: la silueta más alta fue **0.201 con $k = 13$** , Calinski-Harabasz prefirió $k = 2$ y Davies-Bouldin siguió mejorando hasta $k = 15$. Una silueta de 0.20 indica que los grupos se distinguen, pero que muchas recetas viven en la frontera.

8.4.3. ¿Qué encontró?

Con 13 grupos, diez coinciden al 100 % con una etiqueta que ya existía: sopas, desayunos, cerdo, mariscos, bebidas, pollo, res, pavo, cordero y postres. Otro reúne comidas vegetarianas de diario (19 % del catálogo), y los dos restantes salen de la dificultad y no del platillo: uno con las recetas sin dificultad (11 %) y otro con las de reto (1.4 %).

Conclusión: K-Means confirmó que las etiquetas del catálogo son coherentes, pero no descubrió ninguna categoría nueva con lógica de cocina. Por eso no filtra, no ordena y no aparece en la aplicación; las categorías que ve la persona son las etiquetas.

8.5. Asistente generativo con Gemini

— **Para qué sirve.** El asistente acompaña la receta ya elegida: explica un paso, propone una sustitución, convierte una unidad o recuerda una temperatura segura. No sirve para inventar platillos.

El asistente usa Gemini (Google, s.f.), un modelo de lenguaje, guiado por unas instrucciones de sistema¹⁹ y por un conjunto de herramientas:

- **La receta viaja como dato, no como orden.** Si un ingrediente trajera una frase como “ignora tus reglas”, el modelo la trata como texto de la receta.
- **Prohibido inventar** cantidades, pasos, temperaturas o valores nutricionales. Si un dato no existe, el asistente lo dice.
- **Herramientas de cálculo,** porque un modelo de lenguaje redacta bien pero calcula mal: escalar porciones (avisando cuando no se puede escalar parejo, como con la levadura), convertir unidades, sugerir sustitutos según la función del ingrediente y buscar en el catálogo.
- **Temperaturas desde una tabla oficial** del USDA (U.S. Department of Agriculture, s.f.), no desde la memoria del modelo.
- **Alergias con precaución:** puede proponer un cambio, pero nunca afirma que una receta es completamente segura.

Hay pruebas automáticas para las herramientas y para el servicio que arma la respuesta. Lo que todavía falta es un conjunto grande de conversaciones calificadas por personas, que queda en los siguientes pasos.

9. Resolución del problema

9.1. Cómo trabajan juntos el recomendador y el asistente

Las dos piezas se comunican a través de la receta, y cada una tiene su momento: el recomendador actúa *antes* de elegir y el asistente, *después* (Figura 6).



Figura 6: La recomendación ocurre antes de la conversación. El asistente recibe la receta que la persona ya está viendo.

¹⁹Las instrucciones de sistema (*system prompt*) son el texto que define el papel y las reglas del modelo antes de que empiece la conversación.

9.2. Ejemplo de funcionalidad: Con lo que tengo

Caso ilustrativo: una persona con poco tiempo y tres ingredientes

1. Una persona que casi no cocina abre ChefWise con **huevo, tortilla y queso** en el refrigerador y veinte minutos libres. No sabe cómo se llama lo que podría preparar.
2. Entra a **Con lo que tengo** y escribe esos tres ingredientes. Activa el filtro de tiempo.
3. Los filtros duros descartan las recetas que tardan más de lo permitido o que contienen un alérgeno marcado.
4. TF-IDF compara esas palabras con los ingredientes base de cada receta restante y devuelve platillos reales que las usan, indicando cuántos ingredientes extra pide cada uno. Si activa “el menor número de extras”, el orden prioriza aprovechar lo que tiene.
5. Elige una receta. En la ficha ve las cantidades, los pasos y el enlace a la página original.
6. No tiene el chile que pide la receta. Pregunta al asistente con qué cambiarlo; el asistente revisa los ingredientes de *esa* receta, propone un sustituto y explica qué cambia en el sabor.
7. Pregunta si le alcanza para tres personas. La herramienta de escalado ajusta las cantidades y avisa cuáles conviene ajustar con cuidado.

Resultado: en pocos pasos la persona pasó de “no sé qué hacer” a cocinar una receta real con lo que tenía, sin salir de la aplicación.

9.3. Ejemplo de funcionalidad: Sorpréndeme

En Sorpréndeme el camino es otro. La persona marca categorías, dificultad, país y tiempo, y esos filtros son duros. Si marca varias categorías y pocas recetas cumplen todas, la lista se completa con coincidencias parciales y la aplicación lo indica, para que no parezca que el filtro se cumplió al pie de la letra. El texto de esas preferencias pasa por el TF-IDF ya entrenado para ordenar a los candidatos. El botón “otras ideas” excluye las recetas ya mostradas, así la segunda lista no repite la primera.

9.4. Recorrido de la interfaz

La aplicación está construida con Angular en el navegador y una API en FastAPI, publicada en Vercel. El recorrido, pantalla por pantalla, es el siguiente:

1. **Inicio.** Tres caminos: buscar, Sorpréndeme o Con lo que tengo. El botón del asistente está siempre visible.
2. **Sorpréndeme.** Se eligen las fichas del día, el nivel, el país y el tiempo. “Aplicar filtros” devuelve una cuadrícula corta de tarjetas que se pueden guardar, marcar como favoritas u ocultar.
3. **Con lo que tengo.** Se escriben los ingredientes uno por uno, y la búsqueda devuelve recetas que los usan.
4. **Receta.** Se leen ingredientes y pasos. Si hay una alergia marcada y el platillo la contiene, el

aviso aparece en la misma pantalla.

5. **Asistente.** Desde el botón flotante se pregunta por un paso, una sustitución, las porciones o la cocción. La respuesta llega en texto claro y acotada a la receta.
6. **Mi cocina.** Se registran las alergias y se consultan el historial, las favoritas y lo que se quiere cocinar después.

9.5. Cómo responde ChefWise a cada cara del problema

Cuadro 11: Relación entre el problema planteado y la solución construida.

Cara del problema	Respuesta de ChefWise
Indecisión ante muchas opciones	Sorpréndeme: pocos controles, una lista corta y “otras ideas” sin repetir.
Aprovechar la despensa	Con lo que tengo: recetas reales ordenadas por qué tan bien usan lo disponible, con el número de extras.
Poca práctica y dudas al cocinar	Asistente sobre la receta abierta, con herramientas de cálculo y temperaturas del USDA.
Riesgo por alergias	Filtro y aviso visibles; el asistente nunca garantiza que algo es seguro.

10. Conclusiones

Como prueba de concepto, ChefWise cumple los objetivos planteados en la Sección 2: se construyó un catálogo propio de recetas en español, un recomendador que devuelve platillos reales a partir de la despensa o de filtros, un asistente que acompaña la preparación y una aplicación web pública que integra todo. El valor no está en un modelo aislado, sino en el problema que atiende: **elegir y cocinar cuando hay poca práctica, poca compañía y lo que ya se tiene en casa.**

Los hallazgos clave son:

- **Los datos importan tanto como el modelo.** Unir un conjunto público con scraping de Kiwilimón dio un catálogo de 28,798 recetas con fuerte presencia mexicana. Limpiar marcas, letras rotas y tiempos imposibles fue indispensable para que el recomendador comparara platillos y no errores.
- **Un modelo sencillo bien ajustado es suficiente.** TF-IDF pone al menos una receta del mismo tipo en el top 10 en el 93 % de las consultas con receta completa y en el 69 % de las que solo traen ingredientes, frente a menos del 10 % del azar.
- **El modelo más potente no siempre es el que se publica.** Los embeddings ganan con recetas completas, pero empatan con listas de ingredientes y no caben en el despliegue. La decisión coincide con la pantalla más importante: escribir lo que hay en casa.
- **Un resultado negativo también es un resultado.** K-Means, con silueta máxima de 0.201, solo redescubrió las etiquetas existentes. Eso confirmó su calidad y evitó agregar a la interfaz grupos que no aportaban nada.
- **La IA generativa funciona mejor con límites.** El asistente es útil porque está atado a una receta, a herramientas de cálculo y a una tabla oficial de seguridad.

Para mantener la aplicación al día, el proceso se puede repetir cada vez que se quiera ampliar el catálogo:

1. Ejecutar el scraping y descargar la versión más reciente del conjunto público.
2. Aplicar las reglas de limpieza: codificación, marcas, tiempos y registros incompletos.
3. Construir las variables y correr las validaciones automáticas, incluida la alarma de la etiqueta vegetariana.
4. Reentrenar el vectorizador TF-IDF y regenerar la matriz.
5. Repetir las pruebas A y B para confirmar que las métricas se mantienen.
6. Publicar los nuevos archivos en la aplicación.

La solución no es perfecta. Ningún modelo lo es, y reconocer su margen de error deja claras las oportunidades de mejora que se describen a continuación.

10.1. Siguientes pasos

La versión publicada ya recomienda y ya acompaña, pero sigue siendo una prueba de concepto. Para que crezca y se vuelva un producto en el que las personas confíen todos los días, hay cinco frentes de trabajo. No son independientes: la base de datos es el cimiento de los demás, porque guarda las recetas que se van a controlar y las interacciones que necesitan la recomendación híbrida y las pruebas con usuarios. El Cuadro 12 los resume y los siguientes apartados detallan cada uno.

Cuadro 12: Frentes de trabajo, en el orden en que conviene atacarlos.

Frente	Qué resuelve	Cómo se sabrá que funcionó
1. Base de datos	Hoy todo vive en archivos CSV y en el navegador; no hay cuentas ni historial compartido entre dispositivos.	Las preferencias de una persona aparecen en cualquier dispositivo.
2. Control de las recetas	El catálogo se arma a mano y no hay forma de corregir ni de reportar una receta con errores.	Menos recetas incompletas y duplicadas en cada versión del catálogo.
3. Probar otros modelos	Solo se compararon TF-IDF y un modelo de embeddings.	Mejores métricas en las pruebas A y B, sin superar el espacio del despliegue.
4. Recomendación híbrida	El recomendador solo mira el texto de la receta, no lo que las personas eligen y cocinan.	Más recetas abiertas y guardadas por cada lista mostrada.
5. Pruebas con usuarios	No se sabe todavía si la interfaz se entiende a la primera.	Más tareas completadas, en menos tiempo y con menos errores.

10.1.1. Una base de datos para la aplicación

— **Para qué sirve.** Una base de datos²⁰ permite que la información deje de vivir en archivos sueltos y en el navegador de cada persona. Sirve para tener cuentas, recordar la despensa y, sobre todo, para empezar a registrar cómo se usa la aplicación.

²⁰Una base de datos relacional guarda la información en tablas conectadas entre sí por claves. Permite consultar, actualizar y proteger los datos de muchas personas al mismo tiempo, algo que un archivo CSV no puede hacer.

Hoy el catálogo se lee de dos archivos CSV, y las favoritas, el historial y las alergias se guardan en el navegador. Si la persona cambia de teléfono, pierde todo. Y la aplicación no aprende nada de lo que la gente elige. La propuesta es una base de datos relacional con estas tablas principales:

- **Recetas, ingredientes y etiquetas**, en tablas separadas y unidas por `recipe_key`. Así un ingrediente como `jitomate` existe una sola vez y se puede corregir en un solo lugar.
- **Usuarios, alergias y despensa**, para que Con lo que tengo abra con lo último que la persona dijo tener y las alergias viajen con su cuenta.
- **Interacciones**: cada vez que alguien abre, guarda, oculta o marca como cocinada una receta. Esta tabla es la pieza que hoy falta. Sin ella no hay datos de comportamiento, y sin esos datos no se puede entrenar un modelo supervisado ni uno híbrido.

Las alergias son información de salud, así que se guardarían solo con consentimiento, protegidas con cifrado y con la opción de borrarlas en cualquier momento.

10.1.2. Mayor control sobre las recetas

Hoy el catálogo se arma corriendo los notebooks de limpieza y se publica como un archivo nuevo. Funciona, pero no deja rastro de qué cambió entre versiones, ni permite corregir una sola receta sin rehacer todo. Para tener más control se proponen cinco medidas:

1. **Versiones del catálogo.** Cada actualización queda registrada con su fecha, su fuente y las recetas que entraron, cambiaron o salieron. Si una versión introduce errores, se puede regresar a la anterior.
2. **Validaciones automáticas en cada carga.** Las comprobaciones que hoy corren en el notebook (identificadores únicos, textos vacíos, etiquetas coherentes y la alarma de la etiqueta vegetariana) se vuelven una puerta obligatoria: una receta que no las pasa no llega a la base de datos.
3. **Duplicados entre fuentes.** El mismo platillo puede aparecer en dos páginas con nombres distintos. La similitud coseno, que ya se usa para recomendar, también sirve para detectar pares de recetas casi idénticas y revisarlos antes de publicarlos.
4. **Datos que hoy faltan, marcados como tales.** Completar porciones, unidades y tiempos cuando se puedan estimar, siempre con una marca de “estimado” para no presentarlos como dato de la fuente. También guardar la foto que las páginas ya publican, con su crédito y su enlace.
5. **Reportes de las personas.** Un botón de “reportar receta” (un paso confuso, un ingrediente que falta, una etiqueta equivocada) que envíe el caso a una lista de revisión. Las personas que cocinan detectan errores que ninguna regla automática ve.

10.1.3. Probar otros modelos

La comparación de este proyecto fue entre dos enfoques. Quedan alternativas que podrían mejorar la recomendación sin romper el límite de espacio, y conviene evaluarlas con las mismas pruebas A y B y las mismas 320 consultas, para que la comparación sea justa:

- **Variantes de TF-IDF.** Por ejemplo, BM25, que es la fórmula que usan muchos buscadores y que ajusta mejor el peso de las recetas largas y cortas. A eso se puede sumar un diccionario de sinónimos (`jitomate`–`tomate`, `papa`–`patata`, `camarón`–`gamba`) armado con el propio catálogo.

- **Embeddings más ligeros.** Modelos más pequeños o cuantizados²¹ que conserven la ventaja con sinónimos y quepan en el despliegue. Otra opción es dejarlos en un servicio aparte.
- **Otros agrupamientos.** Repetir K-Means sin las columnas de dificultad, y probar algoritmos que no piden el número de grupos por adelantado, como DBSCAN, que además identifica las recetas que no encajan en ningún grupo.
- **Otros modelos de lenguaje para el asistente.** Comparar distintos proveedores con un conjunto fijo de preguntas difíciles: alergias, temperatura del pollo, “invéntame una receta que no esté”, “¿cuántas calorías tiene?”. Se elige el que respete mejor las reglas, no el que redacte más bonito.

10.1.4. Recomendación híbrida

— **Para qué sirve.** Un recomendador híbrido combina dos o más formas de medir qué tan buena es una receta para una persona. Sirve para aprovechar lo mejor de cada modelo y para sumar algo que hoy no existe: lo que las personas realmente eligen.

Hoy ChefWise usa solo **recomendación basada en contenido**: compara el texto de las recetas. Existe otra familia, el **filtrado colaborativo**²², que aprende de lo que hacen las personas. Un modelo híbrido junta las dos con una calificación combinada:

Ejemplo ilustrativo del puntaje híbrido

Con pesos $\alpha = 0.5$, $\beta = 0.3$ y $\gamma = 0.2$, una receta con similitud TF-IDF de 0.40, similitud por embeddings de 0.80 y señal de usuarios de 0.50 obtiene:

$$0.5(0.40) + 0.3(0.80) + 0.2(0.50) = 0.20 + 0.24 + 0.10 = 0.54$$

Si solo se usara TF-IDF, la receta tendría 0.40 y podría quedar detrás de otras. La parte híbrida la sube porque significa lo mismo que la consulta y porque a otras personas les ha servido. Los números son un ejemplo, no resultados del proyecto.

10.1.5. Pruebas con usuarios para mejorar la interfaz

— **Para qué sirve.** Una prueba de usabilidad consiste en observar a personas reales mientras intentan hacer tareas concretas con la aplicación. Sirve para descubrir dónde se confunden, qué no encuentran y qué sobra, cosas que quien construyó la interfaz ya no puede ver.

Hasta ahora la interfaz se probó con pruebas automáticas, que revisan que el código funcione pero no que se entienda. El plan es hacer rondas cortas de prueba:

1. **Participantes.** De cinco a ocho personas por ronda, de los cuatro perfiles de la estrategia: principiantes, quien quiere aprovechar su despensa, quien no se decide y quien cocina con una alergia.
2. **Tareas concretas.** Por ejemplo: “encuentra algo para cenar en 20 minutos con huevo y tortilla”, “marca una alergia al cacahuete y busca un postre”, “pregúntale al asistente con qué

²¹Cuantizar un modelo es guardar sus números con menos precisión (por ejemplo, 8 bits en lugar de 32). El modelo ocupa varias veces menos espacio y suele perder muy poca calidad.

²²El filtrado colaborativo recomienda a partir del comportamiento de muchas personas: si quienes cocinaron A también cocinaron B, a quien cocinó A se le sugiere B. Es la idea detrás de las recomendaciones de plataformas como Netflix o Spotify.

sustituir un ingrediente” y “guarda una receta y vuelve a encontrarla”.

3. **Pensar en voz alta.** Se pide a cada persona que diga lo que piensa mientras usa la aplicación, para entender no solo dónde falla sino por qué.
4. **Métricas.** Porcentaje de tareas completadas, tiempo hasta abrir una receta, número de clics, errores, y un cuestionario corto de satisfacción al final, como la escala SUS²³.
5. **Corregir y repetir.** Se arreglan los problemas más frecuentes y se hace otra ronda, para confirmar que el cambio realmente ayudó.

Con la base de datos en marcha, también se pueden hacer pruebas A/B: mostrar dos versiones de una pantalla a grupos distintos y medir cuál lleva a más recetas abiertas y cocinadas. Por ejemplo, una cuadrícula de opciones contra una sola propuesta con un botón de “otra”. Las respuestas del asistente se calificarían con la misma lógica: ¿la explicación alcanzó?, ¿la sustitución funcionó?, ¿la temperatura coincide con la tabla del USDA?

El orden importa

Primero la base de datos y el control de las recetas, porque todo lo demás se apoya en ellos. En paralelo, la primera ronda de pruebas con usuarios, que no necesita esperar a nada. Después, otros modelos y la recomendación híbrida, que necesitan las interacciones que la base de datos empieza a guardar.

Con estos pasos, ChefWise se acerca todavía más a la pregunta con la que empezó este documento: menos tiempo eligiendo, más aprovechamiento de lo que ya está en casa y una compañía concreta mientras se cocina.

Referencias

- Caliński, T. y Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, 3(1), 1–27.
- Davies, D. L. y Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.
- Efron, B. y Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.
- Google. (s.f.). *Gemini API documentation*. Recuperado el 4 de octubre de 2026, de <https://ai.google.dev/>
- Järvelin, K. y Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.
- Kiwilimón. (s.f.). *Kiwilimón: recetas de cocina* [Sitio web]. Recuperado el 4 de octubre de 2026, de <https://www.kiwilimon.com/>
- MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. En *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* (Vol. 1, pp. 281–297). University of California Press.

²³La escala SUS (*System Usability Scale*) es un cuestionario estándar de diez preguntas que da una calificación de usabilidad de 0 a 100. Permite comparar una versión de la interfaz contra otra.

- Manning, C. D., Raghavan, P. y Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- Salton, G. y Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- Somos NLP. (s.f.). *RecetasDeLaAbuela* [Conjunto de datos]. Hugging Face. <https://huggingface.co/datasets/somosnlp/RecetasDeLaAbuela>
- Spearman, C. (1904). The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1), 72–101.
- U.S. Department of Agriculture, Food Safety and Inspection Service. (s.f.). *Safe minimum internal temperature chart*. <https://www.fsis.usda.gov/food-safety/safe-food-handling-and-preparation/food-safety-basics/safe-temperature-chart>
- Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R. y Wei, F. (2024). *Multilingual E5 text embeddings: A technical report* (arXiv:2402.05672). arXiv. <https://arxiv.org/abs/2402.05672>